

helixtranslate_llmsverifier_integration _structure

HelixTranslate LLMsVerifier Integration Plan — Complete Document Structure

Document Metrics Summary

Metric	Value
Total Target Words	~22,500
Chapters (H2)	8
Sections (H3)	47
Content Points (H4)	142
Estimated Code Blocks	38
Estimated Tables	24
Estimated Diagrams	6

H1: HelixTranslate LLMsVerifier Integration Plan

Title page only. Subtitle: "Enterprise-Grade Model Verification, Discovery, and Scoring — Phase-Based Implementation Guide"

H2: Chapter 1 — Executive Summary & Integration Architecture Overview

Target Word Count: 1,800 | Code Blocks: 2 | Tables: 3 | Diagrams: 1

H3: 1.1 Integration Vision & Goals

H4: Define the single-source-of-truth mandate for model metadata

H4: Map current HelixTranslate provider chaos (9 factories, no validation) to target state

H4: List the three non-negotiable UX guarantees: validated-only, positively-scored, auto-discovered

H3: 1.2 Reference Architecture from HelixAgent

H4: Present the HelixAgent 8-phase startup pipeline diagram

H4: Map HelixAgent's internal/verifier/startup.go phases to HelixTranslate equivalents

H4: Extract the score adapter pattern from internal/services/llmsverifier_score_adapter.go

H3: 1.3 High-Level Component Map

H4: Table: Component mapping — HelixAgent → HelixTranslate (8 components: verifier, scoring, discovery, verification, challenges, adapters, constitution, docs)

H4: Diagram: Data flow from LLMsVerifier discovery → verification → scoring → HelixTranslate provider factory

H4: Table: File-to-file integration mapping with line-number references for all 3 repos

H3: 1.4 Phase Overview

H4: Present the 8-phase timeline table with week allocations, deliverables, and success criteria

H4: Define the gating criteria between each phase (must pass to proceed)

H2: Chapter 2 — Phase 1: Foundation & Repository Preparation

Target Word Count: 2,200 | Code Blocks: 6 | Tables: 2 | Diagrams: 1

H3: 2.1 Dependency Management & Module Configuration

H4: Add LLMsVerifier module import to `go.mod` — exact command and require block

H4: Resolve Go version conflict: HelixTranslate 1.25.2 vs LLMsVerifier 1.25.3 vs HelixAgent 1.26

H4: Run `go mod tidy` and document expected transitive dependencies

H4: Table: Direct dependency audit — before vs after integration

H3: 2.2 Project Structure Creation

H4: Create `internal/verifier/` directory with subdirectories: `scoring/`, `discovery/`, `verification/`, `challenges/`

H4: Create `internal/services/` directory for score adapter

H4: Create `internal/verifier/config/` for verifier-specific configuration structs

H4: Code block: Expected directory tree output from `tree internal/verifier/`

H3: 2.3 Configuration Schema Design

H4: Extend `internal/config/config.go` with `LLMsVerifierConfig` struct — exact lines to modify

H4: Define `.env` variables: `LLMSVERIFIER_ENABLED`, `LLMSVERIFIER_API_URL`, `LLMSVERIFIER_API_KEY`, `LLMSVERIFIER_DB_PATH`, `LLMSVERIFIER_CACHE_TTL`

H4: Add YAML config section under new `verifier:` key with all nested fields

H4: Code block: Complete `LLMsVerifierConfig` struct with JSON tags and validation

H3: 2.4 Interface Alignment

H4: Compare HelixTranslate `pkg/translator/llm/provider.go` `LLMClient` interface with LLMsVerifier provider interface

H4: Define the bridging interface in `internal/verifier/bridge.go`

H4: Code block: Full `VerifiedProvider` interface with method signatures

H4: Document the adapter pattern: how verified models feed into existing pkg/translator/llm/ factory

H3: 2.5 Constitution & Governance Files Setup

H4: Create internal/verifier/CONSTITUTION.md — 33-rule template adapted from HelixAgent

H4: Review and update CLAUDE.md with LLMsVerifier context — exact sections to append

H4: Review and update AGENTS.md with verifier agent instructions — exact sections to append

H4: Table: Rule mapping — HelixAgent constitution rules → HelixTranslate equivalents

H2: Chapter 3 — Phase 2: Core Verification Engine Integration

Target Word Count: 3,000 | Code Blocks: 8 | Tables: 3 | Diagrams: 1

H3: 3.1 Verifier Client Initialization

H4: Implement `internal/verifier/client.go` with `NewVerifierClient()` constructor

H4: Code block: `NewVerifierClient()` with config injection, JWT auth setup, and connection pooling

H4: Integrate verifier client initialization into existing `HelixTranslate` startup sequence

H4: Add health-check endpoint: `internal/verifier/health.go` with `Ping()` method

H3: 3.2 The 8-Step Verification Pipeline Binding

H4: Table: Pipeline step mapping — `LLMsVerifier` step name → `HelixTranslate` handler function → file location

H4: Implement `internal/verifier/pipeline.go` — `VerificationPipeline` struct with 8 ordered steps

H4: Code block: Pipeline definition with step ordering: `existence` → `responsive` → `feature_detect` → `code_visibility` → `coding` → `perf` → `cost` → `validate`

H4: Code block: Per-step error handling and retry logic with exponential backoff

H3: 3.3 Existence Verification Implementation

H4: Bind `LLMsVerifier` existence check to `verifier.Verify()` for model availability

H4: Code block: `VerifyModelExistence(modelID string) (bool, error)` implementation

H4: Cache existence results in SQLite with TTL from config

H3: 3.4 Responsive & Feature Detection

H4: Implement `internal/verifier/responsive.go` — latency threshold checking

H4: Implement `internal/verifier/feature_detect.go` — capability flag extraction

H4: Code block: Feature detection result mapping to `HelixTranslate` capability enums

H4: Table: Feature flags → HelixTranslate provider capability matrix

H3: 3.5 Code Visibility & Coding Challenge Execution

H4: Implement `internal/verifier/code_visibility.go` — context window and code comprehension test

H4: Implement `internal/verifier/coding_challenge.go` — challenge runner interface

H4: Code block: Challenge execution with timeout, sandbox isolation, and result collection

H4: Table: 193+ HelixAgent challenge categories mapped to HelixTranslate translation-relevant subset

H3: 3.6 Performance Benchmarking

H4: Implement `internal/verifier/perf.go` — throughput and latency benchmarking

H4: Define translation-specific benchmarks: tokens/sec, first-token latency, total translation time

H4: Code block: `BenchmarkTranslation(modelID string, sampleTexts []string) (*PerfResult, error)`

H3: 3.7 Cost Analysis Integration

H4: Implement `internal/verifier/cost.go` — per-token cost calculation

H4: Bind LLMsVerifier cost API to HelixTranslate cost tracking

H4: Code block: Cost normalization across 25+ provider pricing models

H2: Chapter 4 — Phase 3: Scoring Engine & Score Adapter

Target Word Count: 2,800 | Code Blocks: 7 | Tables: 4 | Diagrams: 1

H3: 4.1 Scoring Engine Initialization

H4: Implement `internal/verifier/scoring/engine.go` — `ScoringEngine` wrapper around `scoring.NewScoringEngine()`

H4: Code block: Engine constructor with config-driven weight overrides

H4: Document thread-safety: scoring engine is read-heavy, concurrent-safe design

H3: 4.2 Five-Component Score Calculation

H4: Table: Weight configuration — `HelixAgent` defaults → `HelixTranslate` translation-optimized weights

H4: Implement `internal/verifier/scoring/components.go` — individual component calculators

H4: Code block: `CalculateResponseSpeedScore()` — 25% weight, latency-based

H4: Code block: `CalculateCostScore()` — 25% weight, price-per-token based

H4: Code block: `CalculateEfficiencyScore()` — 20% weight, throughput ratio

H4: Code block: `CalculateCapabilityScore()` — 20% weight, challenge pass rate

H4: Code block: `CalculateRecencyScore()` — 10% weight, model age decay function

H3: 4.3 Composite Score & Threshold Management

H4: Implement `internal/verifier/scoring/composite.go` — weighted sum aggregator

H4: Define minimum score thresholds per provider tier (premium ≥ 0.85 , standard ≥ 0.70 , budget ≥ 0.55)

H4: Code block: `IsModelQualified(modelID string, minScore float64) (bool, *CompositeScore, error)`

H3: 4.4 Score Adapter Service

H4: Implement `internal/services/llmsverifier_score_adapter.go` — one-to-one with `HelixAgent` reference

H4: Code block: Full ScoreAdapter struct with Adapt() method converting LLMsVerifier scores to HelixTranslate ProviderScore

H4: Table: Field mapping — LLMsVerifier Score struct → HelixTranslate ProviderScore struct

H4: Code block: Score caching with invalidation triggers

H3: 4.5 Score Persistence & Historical Tracking

H4: Implement internal/verifier/scoring/history.go — SQLite-backed score history

H4: Code block: Schema for model_scores table with timestamp, model_id, component scores, composite score

H4: Implement score trend analysis: improving, stable, declining classification

H2: Chapter 5 — Phase 4: Model Discovery & Registry Integration

Target Word Count: 2,600 | Code Blocks: 6 | Tables: 3 | Diagrams: 1

H3: 5.1 Three-Tier Discovery Service

H4: Implement `internal/verifier/discovery/service.go` — `DiscoveryService` wrapping `discovery.NewService()`

H4: Code block: Three-tier discovery logic: user config → provider API → `models.dev`

H4: Table: Discovery source priority, fallback order, and timeout settings

H3: 5.2 Provider API Model Enumeration

H4: Implement discovery adapters for all 9 HelixTranslate providers: OpenAI, Anthropic, DeepSeek, Zhipu, Qwen, Gemini, Ollama, LlamaCpp, Mock

H4: Code block: Generic provider discovery interface and per-provider implementations

H4: Rate-limiting and pagination handling for provider API enumeration

H3: 5.3 Model Registry & Single Source of Truth

H4: Implement `internal/verifier/discovery/registry.go` — unified model registry

H4: Code block: `ModelRegistry` struct with CRUD operations and search/indexing

H4: SQLite schema for `verified_models` table with all metadata fields

H4: Code block: Registry query API — `ListModels()`, `GetModel()`, `FilterByScore()`, `FilterByProvider()`

H3: 5.4 Verified Model Filtering & Gatekeeping

H4: Implement `internal/verifier/discovery/gatekeeper.go` — model access control

H4: Code block: `Gatekeeper` struct with `IsModelAvailable(modelID)` that checks verification + score threshold

H4: Table: Gatekeeping rules — verification status × score threshold × provider status → availability

H4: Integration point: gatekeeper feeds into `pkg/translator/llm/` factory model list

H3: 5.5 Background Discovery & Sync

H4: Implement `internal/verifier/discovery/sync.go` — periodic background discovery runner

H4: Code block: Cron-based sync with configurable intervals per provider

H4: Event integration: publish `ModelDiscovered`, `ModelUpdated`, `ModelRemoved` events to `pkg/events/` bus

H3: 5.6 API Endpoint: Model Catalog

H4: Add `/api/v1/models` endpoint to `pkg/api/handler.go` — list all verified models

H4: Add `/api/v1/models/{id}` endpoint — detailed model info with scores

H4: Add `/api/v1/models/discover` endpoint — trigger on-demand discovery

H4: Code block: Complete handler implementations with pagination and filtering

H2: Chapter 6 — Phase 5: Provider Factory Integration & Runtime Model Selection

Target Word Count: 2,800 | Code Blocks: 7 | Tables: 3 | Diagrams: 1

H3: 6.1 Verified Provider Factory Refactoring

H4: Modify `pkg/translator/llm/provider.go` — add `VerifiedLLMClient` wrapping `LLMClient` with verification metadata

H4: Code block: `VerifiedLLMClient` struct with embedded `LLMClient` + `VerificationStatus` + `CompositeScore`

H4: Refactor existing 9 provider factories to return `VerifiedLLMClient` instances

H3: 6.2 Runtime Model Selection Engine

H4: Implement `internal/verifier/selection/engine.go` — intelligent model selection

H4: Code block: `SelectModel(task TaskRequirements)` (`*VerifiedLLMClient, error`) with multi-criteria optimization

H4: Selection strategies: best-score, lowest-cost, fastest, balanced (configurable)

H4: Table: Task type → selection strategy mapping (legal, medical, technical, creative, etc.)

H3: 6.3 Fallback & Degradation Chain

H4: Implement `internal/verifier/selection/fallback.go` — ordered fallback chain

H4: Code block: Fallback chain construction from score-sorted model list with provider diversity

H4: Degradation policy: on provider failure, retry same model ×2 → next model → human escalation

H3: 6.4 Event Bus Integration for Model Lifecycle

H4: Publish model lifecycle events to `pkg/events/` bus: `ModelSelected`, `ModelFailed`, `ModelRetired`

H4: Subscribe to verification events: auto-deselect on score drop below threshold

H4: Code block: Event handlers in `internal/verifier/events.go`

H3: 6.5 MCP, LSP, ACP, Embedding, RAG, Skill, Plugin Integration Points

**H4: Table: All 35 MCPs, 10 LSPs, ACP, 13 embeddings, RAGs, skills, plugins
→ model requirements**

H4: Implement capability advertisement: each component declares model requirements

H4: Code block: CapabilityRequirements struct and matching logic against verified model features

H4: Integration: MCP tool selection uses verified model capability flags

H3: 6.6 API Key Management Through Config

H4: Extend internal/config/config.go with per-provider API key validation through LLMsVerifier

H4: Code block: API key test on startup — ValidateAPIKey(provider, key) (bool, error)

H4: Document .env key naming convention: {PROVIDER}_API_KEY format maintained

H2: Chapter 7 — Phase 6: Testing, Quality Assurance & Anti-Bluff Verification

Target Word Count: 3,500 | Code Blocks: 8 | Tables: 4 | Diagrams: 1

H3: 7.1 Testing Strategy Overview

H4: Table: Test pyramid — unit (70%), integration (20%), e2e (10%) with target counts

H4: Table: Current 43.6% coverage gap analysis by package — before vs after targets

H4: Testing philosophy: "anti-bluff" — every test proves real behavior, no mock-only coverage

H3: 7.2 Unit Testing — Core Verification Components

H4: Test `internal/verifier/client.go` — mock `LLMsVerifier` server with `httptest`

H4: Code block: `client_test.go` — constructor, health check, error handling (target: 95% coverage)

H4: Test `internal/verifier/pipeline.go` — each of 8 steps with injected pass/fail

H4: Code block: `pipeline_test.go` — step ordering, error propagation, retry logic

H3: 7.3 Unit Testing — Scoring Engine

H4: Test `internal/verifier/scoring/engine.go` — known-input/expected-output table

H4: Code block: `engine_test.go` — 20 test cases covering all 5 components

H4: Test weight boundary conditions: 0%, 100%, negative (should error)

H4: Test composite score calculation with floating-point tolerance

H3: 7.4 Unit Testing — Score Adapter

H4: Test `internal/services/llmsverifier_score_adapter.go` — field-by-field mapping verification

H4: Code block: `score_adapter_test.go` — `LLMsVerifier` score input → `HelixTranslate` score output

H4: Test cache hit/miss/invalidation scenarios

H3: 7.5 Unit Testing — Discovery & Registry

H4: Test `internal/verifier/discovery/service.go` — mock all 3 tiers

H4: Code block: `discovery_test.go` — tier fallback, timeout handling, deduplication

H4: Test `internal/verifier/discovery/registry.go` — CRUD operations, search, filtering

H4: Test `internal/verifier/discovery/gatekeeper.go` — all availability rule combinations

H3: 7.6 Integration Testing — End-to-End Verification Flow

H4: Implement `tests/integration/verifier_flow_test.go` — full pipeline with dockerized LLMsVerifier

H4: Code block: Docker Compose setup for integration test environment

H4: Test scenario: discover → verify → score → select → translate → validate

H4: Test scenario: model score drops below threshold → auto-removal from available list

H3: 7.7 Integration Testing — API Endpoints

H4: Test `pkg/api/handler.go` new endpoints: `/api/v1/models/*`

H4: Code block: API integration tests with JWT auth, pagination, filtering

H4: Test model selection endpoint with various task requirements

H3: 7.8 Anti-Bluff Challenge Testing

H4: Implement 4 LLMsVerifier-specific challenges in `internal/verifier/challenges/`

H4: Code block: Challenge 1 — Score adapter accuracy challenge (known scores → verify output)

H4: Code block: Challenge 2 — Gatekeeping logic challenge (edge cases → verify availability)

H4: Code block: Challenge 3 — Discovery resilience challenge (provider failures → verify fallback)

H4: Code block: Challenge 4 — Selection fairness challenge (load distribution → verify diversity)

H4: Table: 193+ total challenges — breakdown by subsystem and pass criteria

H3: 7.9 Performance & Load Testing

H4: Implement `tests/perf/verifier_perf_test.go` — scoring engine under concurrent load

H4: Code block: Load test: 1000 concurrent score requests, p50/p95/p99 latency targets

H4: Memory leak detection: 1-hour sustained load test with heap profiling

H4: Table: Performance benchmarks and SLA targets

H2: Chapter 8 — Phase 7 & 8: Documentation, Deployment & Operational Runbook

Target Word Count: 3,800 | Code Blocks: 4 | Tables: 5 | Diagrams: 2

H3: 8.1 Constitution Update — `internal/verifier/CONSTITUTION.md`

H4: Write all 33 mandatory rules adapted for HelixTranslate translation context

H4: Code block: Complete `CONSTITUTION.md` template with placeholder fills

H4: Table: Rule category breakdown — scoring (8), verification (8), discovery (5), selection (5), testing (4), ops (3)

H3: 8.2 CLAUDE.md Update

H4: Append `LLMsVerifier` context to existing `CLAUDE.md` (~12KB) — exact insertion point at line reference

H4: Document: architecture context, key files, verification pipeline, scoring weights, troubleshooting

H4: Code block: Complete `CLAUDE.md` diff showing additions

H3: 8.3 AGENTS.md Update

H4: Append verifier agent instructions to existing `AGENTS.md` (~20KB) — exact insertion point

H4: Define 3 agent roles: discovery agent, verification agent, scoring agent

H4: Code block: Agent instruction templates with context window constraints

H3: 8.4 Submodule Documentation

H4: Table: All submodules (Challenges, Containers) — documentation status checklist

H4: Write `internal/verifier/README.md` — complete API reference for the verifier package

H4: Write `internal/verifier/challenges/README.md` — challenge catalog with descriptions and pass criteria

H3: 8.5 User-Facing Documentation

H4: Write `docs/llmsverifier-integration.md` — high-level integration overview for users

H4: Table: Environment variables reference with descriptions, defaults, and validation rules

H4: Document the model selection UX: how users see only verified, positively-scored models

H4: Write troubleshooting guide: 10 common issues and resolutions

H3: 8.6 Operational Runbook

H4: Table: Monitoring metrics — what to watch, thresholds, alert definitions

H4: Document log analysis: key log patterns in `internal/verifier/logging.go`

H4: Runbook: "Model score dropped below threshold" — detection, diagnosis, remediation steps

H4: Runbook: "Discovery service failure" — fallback procedures and manual model registration

H4: Runbook: "New provider onboarding" — adding the 10th provider to the verified pipeline

H3: 8.7 Deployment Configuration

H4: Docker Compose extension: add LLMsVerifier service to existing HelixTranslate compose

H4: Code block: `docker-compose.verifier.yml` with SQLite volume, network, health checks

H4: Kubernetes manifests: `verifier-deployment.yml`, `verifier-service.yml`, `verifier-configmap.yml`

H4: Table: Resource requirements — CPU, memory, storage per environment (dev/staging/prod)

H3: 8.8 Migration & Rollback Plan

H4: Migration script: `scripts/migrate-to-verifier.sh` — feature flag gradual rollout

H4: Table: Rollout phases — 5% → 25% → 50% → 100% with duration and exit criteria

H4: Rollback procedure: immediate switch to legacy provider factory via feature flag

H4: Data migration: existing model configs → verified model registry mapping

H3: 8.9 Final Checklist & Sign-Off

H4: Master checklist: 50 items across all 8 phases with status, owner, and sign-off

H4: Quality gates: 100% test coverage, all 193+ challenges passing, all docs complete

H4: Definition of Done: final criteria for marking integration complete

Appendix A: Complete File Inventory

#	File Path	Purpose	Phase	Lines (est.)
1	internal/verifier/client.go	Verifier client	2	120
2	internal/verifier/pipeline.go	8-step pipeline	3	200
3	internal/verifier/scoring/engine.go	Scoring engine	4	180
4	internal/verifier/scoring/components.go	Component calculators	4	250
5	internal/verifier/scoring/composite.go	Score aggregator	4	100
6	internal/verifier/scoring/history.go	Score persistence	4	150
7	internal/verifier/discovery/service.go	Discovery service	5	200
8	internal/verifier/discovery/registry.go	Model registry	5	220
9	internal/verifier/discovery/gatekeeper.go	Access control	5	120
10	internal/verifier/discovery/sync.go	Background sync	5	130
11	internal/verifier/selection/engine.go	Model selection	6	180
12	internal/verifier/selection/fallback.go	Fallback chain	6	140
13	internal/services/llmsverifier_score_adapter.go	Score adapter	4	160
14	internal/verifier/bridge.go	Interface bridge	2	80

#	File Path	Purpose	Phase	Lines (est.)
15	internal/verifier/health.go	Health checks	2	60
16	internal/verifier/config.go	Verifier config	2	100
17	internal/verifier/events.go	Event handlers	6	110
18	internal/verifier/CONSTITUTION.md	Governance rules	7	150
19-52	internal/verifier/challenges/*.go	4 challenge files + tests	6	800
53-70	*_test.go files (18 files)	Test files	6	2400
71	pkg/api/handler.go (modified)	API handlers	5	+120
72	pkg/translator/llm/provider.go (modified)	Provider interface	6	+60
73	internal/config/config.go (modified)	Config struct	2	+40

Appendix B: Glossary

Term	Definition
LLMsVerifier	The external Go module (digital.vasic.llmsverifier) providing model verification, scoring, and discovery
Verified Model	A model that has passed all 8 verification pipeline steps
Composite Score	Weighted sum of 5 component scores (speed, cost, efficiency, capability, recency)
Gatekeeper	Component that controls model availability based on verification status and score thresholds
Discovery Service	Three-tier system for finding available models across providers
Score Adapter	Translation layer between LLMsVerifier score format and HelixTranslate internal format
Anti-Bluff Test	A test that validates real system behavior, not just mocked interactions
Challenge	A structured validation test with defined inputs, expected behavior, and pass criteria

Appendix C: Cross-Reference Index

HelixAgent Reference File	HelixTranslate Target File	Purpose
internal/verifier/startup.go	internal/verifier/client.go	Client initialization
internal/verifier/scoring.go	internal/verifier/scoring/engine.go	Scoring engine
internal/verifier/enhanced_scoring.go	internal/verifier/scoring/components.go	Component scoring
internal/verifier/discovery.go	internal/verifier/discovery/service.go	Discovery service
internal/verifier/verification.go	internal/verifier/pipeline.go	Verification pipeline
internal/services/llmsverifier_score_adapter.go	internal/services/llmsverifier_score_adapter.go	Score adapter (same name)
CONSTITUTION.md	internal/verifier/CONSTITUTION.md	Governance rules

End of Document Structure — 8 Chapters, 47 Sections, 142 Content Points, ~22,500 words